

Clavier du premier iPhone

Introduction au Machine Learning - Etude de cas

Theo Lopes Quintas

BPCE Payment Services,
Université Paris Dauphine

2026

Contexte

Le téléphone mobile en 2007



(a) Motorola C139



(b) Nokia 1200



(c) Samsung SGH E250



(d) BlackBerry

Figure – Téléphones mobile les plus vendus en 2006-2007

Contexte

Le téléphone mobile en 2007

En 2005, Apple se lance dans le **Project Purple 2** : la conception de l'iPhone. Le marché est bientôt saturé et avec des relais de croissances clairs.

On s'intéresse ici à un problème en particulier : le clavier. D'après les calculs, la taille de l'écran ne permet pas d'avoir une écriture confortable avec un clavier *classique*.

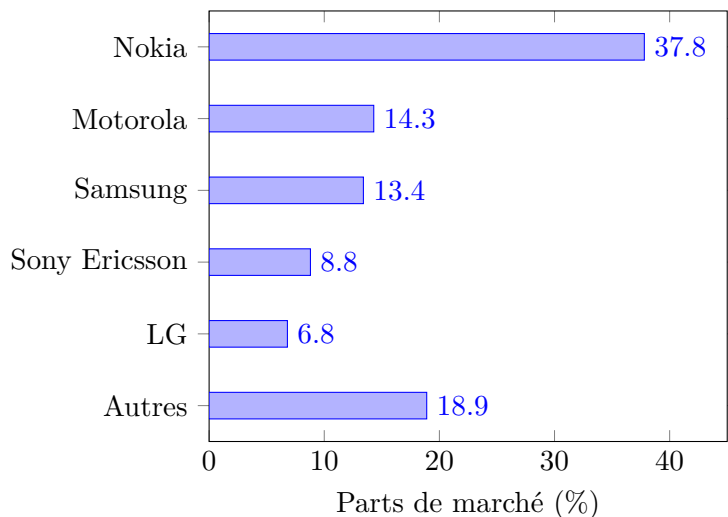


Figure – Parts de marché en 2007 pour les téléphones mobiles

Contexte

Les premiers prototypes

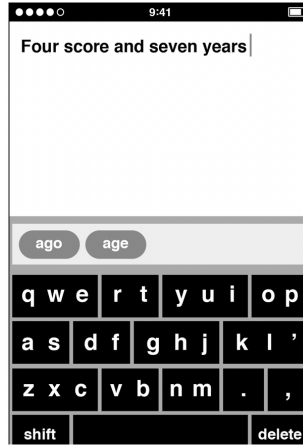
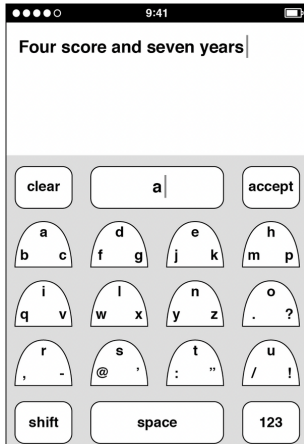
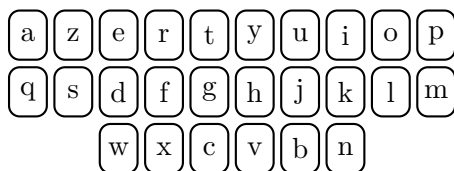


Figure – Deux exemples de prototype réalisé par Ken Kocienda [Kocienda, 2018]

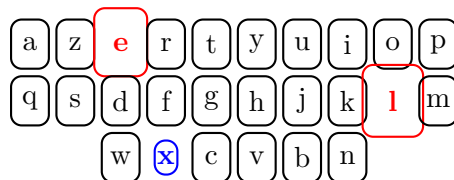
Contexte

Notre solution

On propose d'agrandir virtuellement la taille des touches qui sont les plus probables d'être tapées, en se basant sur l'historique.



(a) Vue de l'utilisateur



(b) Taille réelle de la touche

Figure – Taille dynamique des touches du clavier en fonction de la probabilité de la prochaine lettre

Comment prédire la prochaine touche ?

Prédire plusieurs classes

Première approche : Un contre tous

Nous avons un dataset de features X ainsi qu'une cible $y \in \llbracket 1, K \rrbracket$. On se propose d'entraîner K modèle de classification dont l'objectif sera d'apprendre une unique classe à chaque fois.



Chaque modèle produit une probabilité estimée, mais **tous ensemble** ne le font pas. Il existe des manières pour normaliser un vecteur et faire qu'il ressemble à une distribution, mais il serait plus juste d'apprendre directement la distribution.

Prédire plusieurs classes

Régression Softmax

Dans le cadre d'une régression logistique, nous avons vu que l'on modélise :

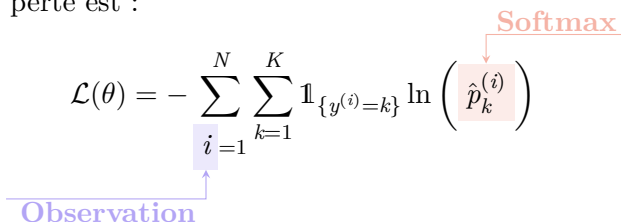
$$\mathbb{P}(Y = 1|x) = \sigma(\langle \theta, x \rangle) = \frac{1}{1 + e^{-\langle \theta, x \rangle}} \quad (\text{Régression logistique})$$

Avec K classes, nous avons donc K vecteurs de poids θ et on note $z_k = \langle \theta_k, x \rangle$. Cette fois, on modélisera comme :

$$\mathbb{P}(Y = k|x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad (\text{Régression Softmax})$$

Dans ce cas, la fonction de perte est :

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \sum_{k=1}^K \mathbb{1}_{\{y^{(i)}=k\}} \ln \left(\overset{\text{Softmax}}{\hat{p}_k^{(i)}} \right)$$



Prédire plusieurs classes

Mesures de la performance d'une classification multiple

Dans le cadre d'une régression logistique, la matrice de confusion pour un seuil fixé est la source de toute les métriques. Adaptée à la classification multiple :

		Prédit					
		a	b	c	...	y	z
Réal	a				...		
	b				...		
	c				...		
	⋮	⋮	⋮	⋮	⋱	⋮	⋮
	y				...		
	z				...		

Figure – Matrice de confusion pour notre problème

Les métriques classiques sont directement calculable pour chaque classe séparément. Pour avoir une valeur unique il y a trois options :

- **Macro** : moyenne arithmétique pour chaque classe
- **Weighted** : moyenne arithmétique pondérée pour chaque classe, en fonction de sa fréquence
- **Micro** : agrégation de toute les classes, puis moyenne pour avoir la métrique moyenne

Il existe également la *top-k accuracy* : voyons-là en pratique !

En résumé

- 1 Contexte 1
- 2 Prédire plusieurs classes 6
 - 2.1 Première approche : Un contre tous 6
 - 2.2 Mesures de la performance d'une classification multiple 8

Bibliographie I



[Kocienda, K. \(2018\).](#)

Creative Selection : Inside Apple's Design Process During the Golden Age of Steve Jobs.

St. Martin's Press.